

PROJECT WORK NCIS

Prof. Giorgio Ventre
a.a. 2025-26

GIACOMO IENGO DE9000048

September 4, 2026

 [GitHub repo](#) 

1 The Idea

Inspired by the paper [1], I implemented the essential aspects of Moving Target Defense ¹ (MTD) within an emulated Mininet network, utilizing OF-switches and a centralized SDN controller. Unlike the topology used by the authors, which could be seen a bit as educational, I tried to immerse MTD in a more conventional environment, which (with a bit of effort) might be seen as the network infrastructure of the DIETI department at University Federico II.

2 Topology and Goal

In Fig.1 we have two autonomous systems; we can imagine that **AS1** is UniNa's one while **AS2** represents the GARR network. Both **R1** and **R2** act as border routers for their corresponding AS, and to emulate the fact that they both run BGP, I used static routes, so UniNa's border router could advertise its two public IP blocks, dedicated to its internal DMZ subnet.

Router **R1** has three interfaces:

- *eth0*: its WAN interface connected to the other AS.
- *eth1*: the interface responsible for the DMZ LAN; here we have the hosts accessible from inside and outside. Every packet destined to 192.168.1.0/24, 192.168.10.0/24, 143.225.1.0/24, and 143.225.10.0/24 will be routed on *eth1*. Within this LAN there are a DNS, a Database, and a WebServer, which can be imagined as the services exposed by the DIETI dept.
- *eth2*: the interface responsible for the NATted LAN; here we have two workstations that are not accessible from outside. They can be seen as two internal offices of the dept. Every packet destined to 192.168.2.0/24 and 192.168.20.0/24 is routed on *eth2*.

Router **R2** has two interfaces:

- *eth0*: its WAN interface connected to the other AS.
- *eth1*: the interface on which sits an everyday user. Although it is unusual that a user directly connects to a border router, this is just a simplification to represent a remote user which connects to the DMZ LAN (prefix 143.225) from outside. This could be seen as a student using DIETI services from home.

This topology resembles a more conventional environment, so the goal is: *given this already working setup, is it possible to implement MTD in DIETI's subnets only by substituting the conventional switches (i.e. S1 and S2) with OF ones? Can a protocol be defined and implemented by a centralized SDN controller?*

¹Moving Target Defense is a technique in the field of SDN networks that involves the continuous change of host IPs. This helps reduce the attack surface, since after an automatic discovery of hosts, it denies a subsequent attack to them because time will elapse between the scanning and the actual attack, so the IPs are likely to be already changed.

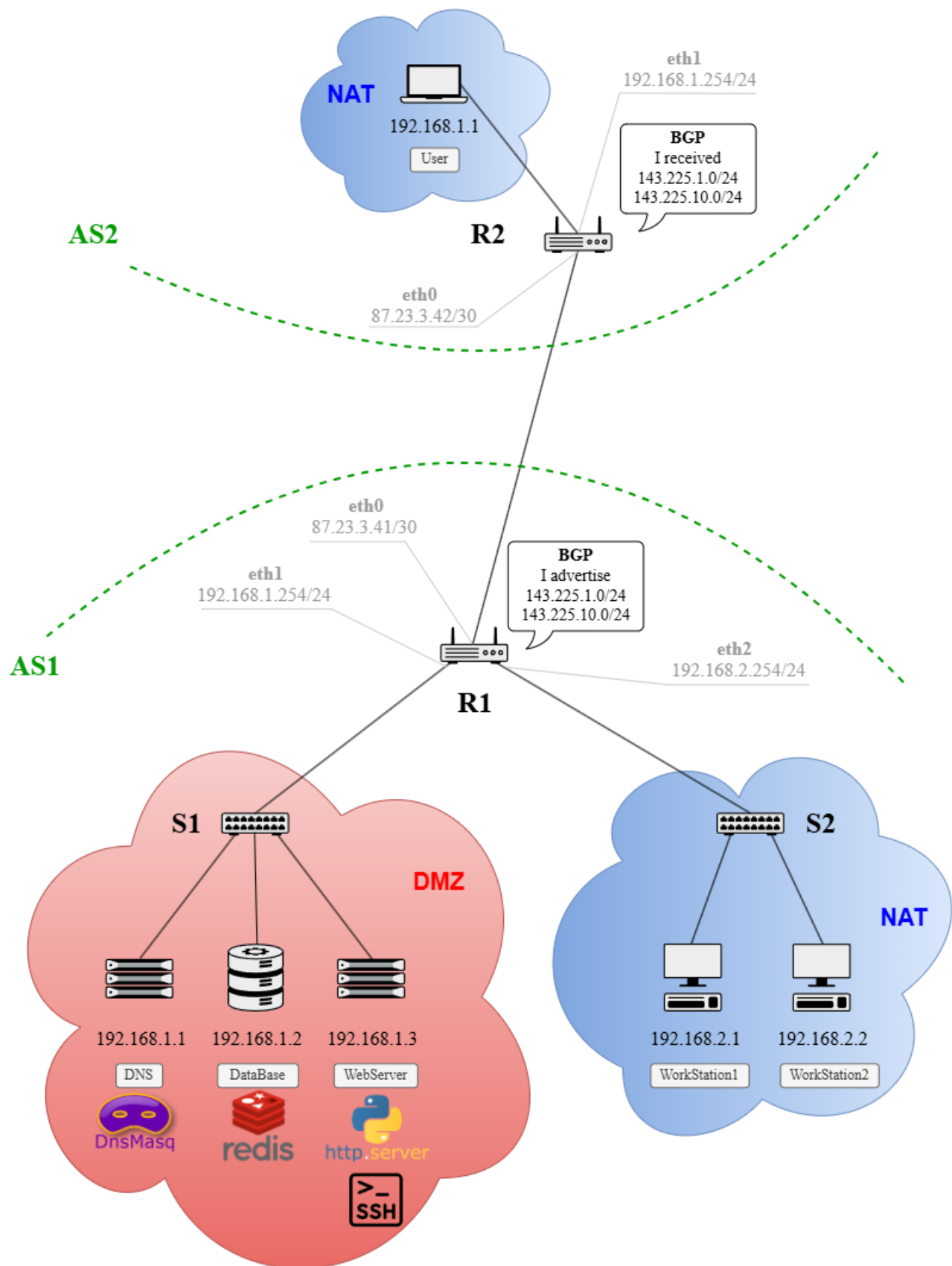


Figure 1: Topology

3 The LAN Protocol

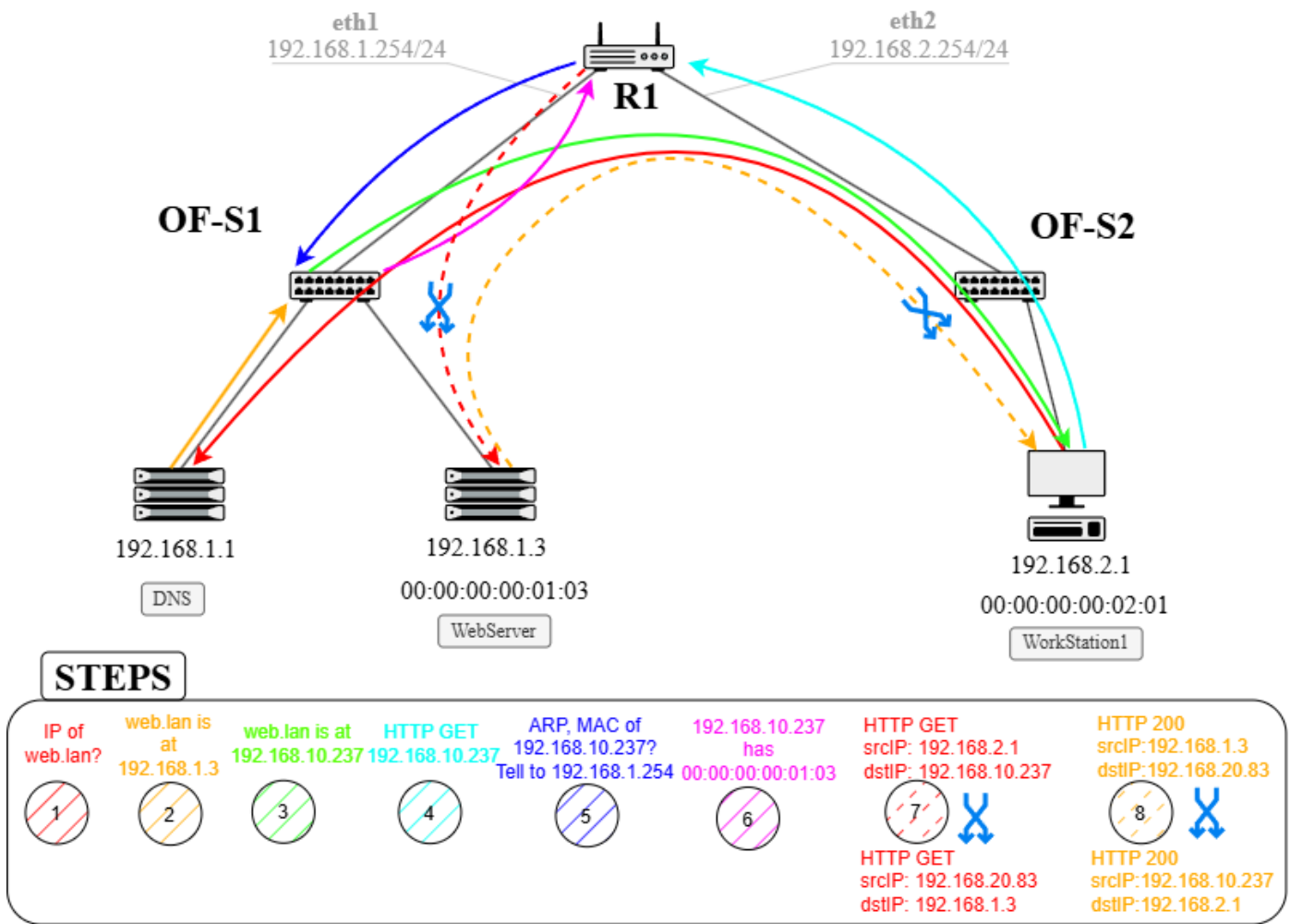


Figure 2: LAN Protocol

Fig.2 shows how MTD is enabled among local hosts. So, the *DNS* is always reachable at 192.168.1.1 because the OF-switches allow normal forwarding to that real IP. Hence (1) *WorkStation1* can query it for *web.lan* since it wants to reach the webserver. (2) The *DNS* answers normally with the real static IP of *WebServer*, but this reply is intercepted (3) and spoofed by **OF-S1**, which will provide the "fake" virtual IP (192.168.10.237) of *WebServer* to *WorkStation1*. At this point (4) *WorkStation1* sends an HTTP request to the virtual IP 192.168.10.237, and since it is not on its own subnet, the packet is forwarded to its gateway 192.168.2.254. The router **R1** already knows that every packet for the subnet 192.168.1.0/24 and 192.168.10.0/24 (the virtual counterpart) has to be forwarded on *eth1*, but here, it doesn't know the MAC address of 192.168.10.237, (5) so **R1** sends an ARP request for that. (6) The ARP request is again intercepted by **OF-S1**, which surely knows the correspondence 192.168.10.237 → 192.168.1.3 and thus is able to answer by providing the MAC address of *WebServer*. At this point **R1** forwards the HTTP request directly to *WebServer*'s MAC, through **OF-S1**, but the destination IP is still virtual so *WebServer* won't be able to process the packet. So, when the HTTP request hits **OF-S1** (7) an InBound translation rule is installed there: "every packet originated by 192.168.2.1 and destined to 192.168.10.237, must be translated so it originates from 192.168.20.83 (*WorkStation1*'s virtual IP) and is destined to 192.168.1.3 (*WebServer*'s real IP)". This way all the packets from *WorkStation1* will reach *WebServer*, but also *WebServer* will be forced to answer to *WorkStation1*'s virtual IP. In fact, the HTTP response will face the exact same treatment on the way back, that is: *WebServer* tries to reach 192.168.20.83, but this is outside of its network, so it goes to **R1**. Then **R1** does an ARP request for the 192.168.20.83 MAC address (since it's looking for the return destination), which will be spoofed by **OF-S2**, and finally a mirrored InBound translation rule (8) will be installed on **OF-S2**. This way the full-duplex connection between the virtual IPs is established.

4 The WAN Protocol

While the LAN protocol enabled connections through virtual IPs among LAN hosts, the WAN protocol must allow the remote user to connect to DMZ hosts through virtual IPs. The mechanism is basically the same as before, so another figure isn't really needed.

It all starts with the user doing a DNS query for `web.public` (*WebServer*). The DNS, as before, is always statically reachable at the public address `143.225.1.1` so the remote user can contact it. The names held by the DNS server are:

```
192.168.1.2 db.lan
192.168.1.3 web.lan
192.168.2.1 ws1.lan
192.168.2.2 ws2.lan
143.225.1.2 db.public
143.225.1.3 web.public
```

So:

1. DNS answers with `143.225.1.3`;
2. **OF-S1** intercepts the answer and spoofs it `143.225.1.3` $\rightarrow$ `143.225.10.189`;
3. User receives `143.225.10.189` and sends an HTTP request there;
4. **R1** receives the HTTP request and knows to forward it on its *eth1*;
5. **R1** does an ARP request: "who has `143.225.10.189`?";
6. **OF-S1** intercepts the request and spoofs the answer, providing the MAC address of *WebServer*;
7. **R1** forwards the HTTP request to *WebServer* but the destination IP is still virtual (and also public this time);
8. An InBound translation rule is installed on **OF-S1**:
"*dstIP*: `143.225.10.189` $\rightarrow$ `192.168.1.3` and *srcIP*: `87.23.3.42` unchanged";
9. An OutBound translation rule is installed on **OF-S1**:
"*srcIP*: `192.168.1.3` $\rightarrow$ `143.225.10.189` and *dstIP*: `87.23.3.42` unchanged";

5 How to run it

1. install ubuntu22.04LTS in a VM
2. `git clone https://github.com/giovannistanco/NCIS_unina.git`
3. `cd NCIS_unina/2026/iengo_giacomo/src`
4. `sudo bash install.bash`
5. run two terminals inside src
 - (a) first terminal: `./run_controller.bash controller.py`, let it start, then
 - (b) second terminal: `./run_topology.bash topology.py`

Important:

- Pressing ENTER on the controller's terminal while it is running will dump its internal state.
- To stop the controller, press CNTRL+C twice.
- To stop mininet is important to exit gracefully typing `mininet> exit`, since the python script will also run after the exit, cleaning up the environment.

6 How to test it

- Remote *User* contacts the DNS
mininet> user dig db.public +short
- *WorkStation1* contacts the DNS
mininet> ws1 dig db.lan +short
- Remote *User* contacts the *WebServer*
mininet> user curl web.public
- *WorkStation1* contacts the *WebServer*
mininet> ws1 curl web.lan
- Remote *User* accesses the SSH server running on *WebServer* (pswd is webuser)
mininet> user ssh webuser@web.public
- *WorkStation1* accesses the SSH server running on *WebServer*
mininet> ws1 ssh webuser@web.lan
- Remote *User* sets a redis key
mininet> user redis-cli -h db.public -p 6379 set mykey "MTD_is_awesome"
- *WorkStation2* retrieves the key
mininet> ws2 redis-cli -h db.lan -p 6379 get mykey
- *WorkStation1* cannot contact the *WebServer* directly
mininet> ws1 curl 192.168.1.3

7 The resulting countermeasure

This technique is mainly a countermeasure against worms, which usually automatically discover hosts and their vulnerabilities and then try to run known exploits against them. This setup proposes to trick the worm into retrieving the host IPs, forcing it to run an attack on them when they're already expired. So the test goes like this: *suppose that WorkStation2 is compromised, meaning the worm can run whatever it wants, and it starts a host discovery.*

There are many techniques in Nmap for host discovery; the main ones are PING, ARP, and TCP (SYN) discovery, used with flags `-sn`, `-sn -PR`, and `-sn -PS` respectively. So the worm runs all of them on the virtual DMZ subnet, where there are basically 2 virtual hosts: *WebServer* and *DataBase*.

```
mininet> ws2 nmap -sn 192.168.10.0/24
Nmap scan report for 192.168.10.7
Host is up (0.70s latency).
Nmap scan report for 192.168.10.78
Host is up (1.2s latency).
Nmap scan report for 192.168.10.83
Host is up (0.18s latency).
Nmap scan report for 192.168.10.203
Host is up (0.013s latency).
Nmap done: 256 IP addresses (4 hosts up) scanned in 71.96 seconds
```

```
mininet> ws2 nmap -sn -PR 192.168.10.0/24
Starting Nmap 7.80 ( https://nmap.org ) at 2026-09-01 16:10 CEST
Nmap scan report for 192.168.10.1
Host is up (0.0083s latency).
Nmap scan report for 192.168.10.84
Host is up (0.95s latency).
Nmap scan report for 192.168.10.131
Host is up (0.79s latency).
Nmap scan report for 192.168.10.190
Host is up (0.0046s latency).
Nmap done: 256 IP addresses (4 hosts up) scanned in 28.17 seconds
```

```
mininet> ws2 nmap -sn -PS 192.168.10.0/24
Starting Nmap 7.80 ( https://nmap.org ) at 2026-09-01 16:11 CEST
Nmap scan report for 192.168.10.111
Host is up (0.0064s latency).
Nmap scan report for 192.168.10.229
Host is up (0.014s latency).
Nmap done: 256 IP addresses (2 hosts up) scanned in 25.68 seconds
```

Notice how the slowest scans actually discover 4 hosts instead of 2. That's because the IP rotation, which occurs every $T = 15$ seconds, happens multiple times during the scan. So in those cases, the worm is not only tricked into discovering IPs that will be useless in a matter of seconds, but also into thinking there are more hosts than there actually are. The fastest scan succeeds in identifying just two hosts, but since 25.68 seconds have elapsed, those IPs will also be useless in a matter of seconds.

This technique is also useful for remote malicious users who want to know if there are vulnerable hosts inside a public IP block:

```
mininet> user nmap -sn -PS 143.225.10.0/24
Starting Nmap 7.80 ( https://nmap.org ) at 2026-09-01 18:26 CEST
Nmap scan report for 143.225.10.85
Host is up (1.0s latency).
Nmap scan report for 143.225.10.125
Host is up (0.0048s latency).
Nmap scan report for 143.225.10.246
Host is up (0.013s latency).
Nmap done: 256 IP addresses (3 hosts up) scanned in 47.24 seconds
```

Here again, by the same mechanism, the scan finds 3 hosts instead of 2.

8 Future improvements

1. As I said at the beginning of this document, I implemented an essential (and slightly different) version of MTD compared to what was proposed by the authors of [1]. In fact, their controller automatically assigns a different mutation interval T_i for every LAN host i , and it calculates ranges for those hosts based on the unpredictability desired for a specific host. My implementation has a standard rotation interval for all hosts and hardcoded IP ranges.
2. As the authors of [1] point out, if the attacker is sophisticated, they could understand what's going on and just contact the hosts directly through the DNS. However, abnormally frequent DNS requests could be a great starting point for pattern recognition and intrusion detection.
3. This technique was proposed by the authors [1] to be fully transparent to the end hosts. This is a bit of a problem because when someone instantiates a connection to an MTD host, that connection has to stay alive even after the subsequent IP mutations, meaning that specific virtual address must still be reachable. This implies the existence of a garbage collection mechanism that cleans up the old vIP connections, but the authors didn't really explain how they handled this. In my implementation, if a virtual IP is unused by a certain host for more than 10 seconds, that vIP expires and is forgotten, thus the UDP/TCP connection is killed (you can experience this by opening an SSH session and waiting for 10 seconds). So there's still a lot of room for improvement to achieve real transparency.

References

- [1] Jafar Jafarian, Ehab Al-Shaer, and Qi Duan. "OpenFlow random host mutation: Transparent moving target defense using software defined networking". In: (Aug. 2012). DOI: 10.1145/2342441.2342467.